shmget

shmat(s

Operating Systems – Midterm Examination
April 17, 2012

Total: 101 points

shmdt

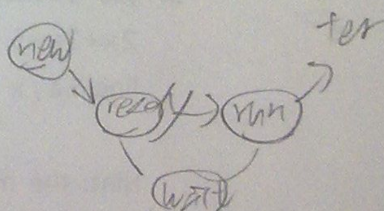
shmctl(IPC_RMID, NULL, 0)

1. Computer system operations:
   (a) (5 points) What do we mean I/O devices and the CPU execute concurrently?
   (b) (10 points) Describe the process of hardware interrupt handling. You need to mention device controller, interrupt-request line, state saving, interrupt vector, and interrupt-specific handler.
   (c) (5 points) What is time-sharing? Explain the difference between it and multi-programming.
   (d) (5 points) What is dual-mode? How does it ensure the proper execution of operating systems?

2. System calls:
   (a) (10 points) How does an API function invoke a system call? You need to mention software interrupt, dual-mode, system call number (index), and table of code pointer.
   (b) (5 points) Explain why programs mostly access system services via API functions rather than making system call directly.

3. (5 points) Describe the pros and cons of the microkernel system structure.

4. Show the state of a process:



   (a) (3 points) after it is selected by a long-term scheduler
   (b) (3 points) before it is selected by a short-term scheduler
   (c) (3 points) after it is selected by a short-term scheduler
   (d) (3 points) before it invokes a blocking system call
   (e) (3 points) after it invokes a blocking system call
   (f) (3 points) after the blocking system call is complete
   (g) (3 points) after it is interrupted by a hardware interrupt

5. (5 points) Show the output of the following program

```
int main(int argc, char *argv[])
{
        pid_t pid;
        int i = 0;

        pid = fork();
i=5     i = i + 5;

        if(pid == 0) {
                i = i + 5;
i=10            printf(“i = %d\n”,i);
        } else {
                wait(NULL);
                printf(“i = %d\n”,i);      2X
        }

        pid = fork ();
        i = i + 5;

        if(pid == 0) {
                i = i + 5;
                printf(“i = %d\n”,i);
        } else {
                wait(NULL);
                printf(“i = %d\n”,i);
        }

        return 0;

}
```

6. (15 points) Please complete the following program that calculates equation 2x+3y-5. The values of variables x and y are provided by users in the command line, i.e., `x = atoi(argv[1])` and `y = atoi(argv[2])`:

Hint: the main program first allocates a shared memory and then creates two child processes - one for the x term calculation and one for the y term calculation. Save the results calculated in the child processes in the shared memory and print the final outcome in the parent process.

2x+3y-5

```
int main(int argc, char *argv[])
{
    int x = atoi(argv[1]);    // the value of variable x
    int y = atoi(argv[2]);    // the value of variable y
    int i;
    pid_t pid;
    const int sh_size = 4;    // shared memory size - 4 bytes

    // allocate a shared memory segment
    int segment_id = shmget(IPC_PRIVATE, sizeof(int), S_IRUSR | S_IWUSR)
    // attach the shared memory segment
    int *shared_memory = (int *) shmat(segment_id,

    *shared_memory = 0;  // initialization

    for(i = 2; i > 0; i--) // create two child processes
    {
       pid = fork()

       if(pid < 0) {
          fprintf(stderr, "Fork Failed");
          exit(-1);

       } else if(pid == 0) { // child part
          if(     )  // do x term calculation

          if(     )  // do y term calculation


          // detach the shared memory segment and exit
          shmdt(shared_memory);

       } else { // parent part

       }
    }

    printf("the result is %d\n",*shared_memory);

    // detach the shared memory segment

    // remove the shared memory segment

    return 0;

}
```

7. Multithread model:
    (a) (5 points) Describe the pros and cons of the many-to-one thread model.
    (b) (10 points) Describe the operation of scheduler activation.